LatinCy word2vec Evaluation Report

Patrick J. Burns
Preprint
Last updated: 9.15.2025

Abstract

A collection of static word embeddings models, i.e. LatinCy word2vec, has been published as part of a larger development plan for LatinCy, pretrained spaCy pipelines for Latin. This report discusses the development of evaluation tasks, specifically an analogy task and an odd-one-out task, for these models and the corresponding evaluation datasets. Results show that the best performing models on both tasks at this point in the development cycle are 300-length vectors trained with the word2vec continuous-bag–of-words (CBOW) algorithm.

Keywords

Latin, natural language processing, word embeddings, word2vec, evaluation

Introduction

Static word embeddings can be a simple yet effective representation of text data for a number of natural language processing tasks and, for this reason, they have become a staple of Latin computational philology.[1] Important foundational work in this area has been done by Rachele Sprugnoli and researchers with the CIRCSE project at Università Cattolica del Sacro Cuore.[2] Examples of Latin literary critical or philological research employing static embeddings models include (but it certainly not limited to) Bjerva and Praet 2015; Manjavacas, Long, and Kestemont 2019; Burns et al. 2021. Yet there has not emerged a clearly preferred set of reference embeddings, as has perhaps become the case for the "Google News" vectors,[3] which is perhaps another way of saying that there is still no consensus around the data sources, parameterization,

[1] An overview of vectors for Latin philological tasks will be available in the following chapter, Burns forthcoming.
[2] Sprugnoli, Passarotti, and Moretti 2019; Sprugnoli, Moretti, and Passarotti 2020.
[3] https://code.google.com/archive/p/word2vec/.

training, etc. on which some kind of standard set of Latin static embeddings model or models could be developed.

In addition, and not unrelated, there is no clear evaluation framework for Latin static word embeddings. Evaluation was highlighted, for example, in Sprugnoli, Passarotti, and Moretti 2019 (further pursued in Burns et al. 2021) with the inclusion of a "synonym selection task." To address this, this report describes work in developing evaluation datasets primarily for use in the development of LatinCy, pretrained spaCy pipelines for Latin.[4] Because of the availability of large amounts of textual data required for training LatinCy, as well as the direct utility of vectors in training specific LatinCy components, such as the lemmatizer, POS tagger, dependency parsers, among others, it has become both straightforward to develop and internally valuable to possess a collection of variously parameterized embeddings models and moreover to know their relative performance.

This report looks specifically at that assessment of relative performance and does so specifically by proposing two evaluation tasks for use with a collection of LatinCy word2vec models.[5] At the same time, since these models are trained on a large collection of Latin text and may be broadly useful to the community of Latin computational philologists, we also publish the evaluation datasets.[6] We report below current model performance and in doing so hope to establish a benchmark guiding further model refinements, both for LatinCy and for the community in general.

Note that in publishing this benchmark, we see it as a starting point for an expanded role of evaluation with respect to this kind of computational philological resource: as will be discussed below, the evaluation datasets published at this time work with a small number of high frequency words in a limited number of categories. Model robustness, that is the ability to serve a wide

[4] Burns 2023. On the spaCy platform more generally, see Honnibal and Montani 2023.

[5] The LatinCy word2vec models are available here: https://huggingface.co/latincy/latincy-w2v. The models are named such that they can be compared by parameterization, e.g. the model *latincy_w2v-CBOW-300-10_0_0_2* can be read as a word2vec model ("w2v") trained using the "CBOW" algorithm with a window of "10"; it is also version 0.0.2 of the model.

[6] The Latincy word2vec evaluation datasets are available here: https://github.com/diyclassics/latincy-w2v-eval/tree/main.

variety of Latin philological tasks and generalize well to different genres, text types, and so on, will only emerge from iterative improvement of future models on existing benchmarks coupled with similar iterative improvement on the evaluation datasets.

Method

Two tasks were considered for this initial phase of evaluation for LatinCy w2v: analogy resolution and odd-one-out selection.[7] As noted above, to foster comparability between the different model types and parameters around, all of the LatinCy w2v models have been trained on the same collection of texts: roughly 273.6M tokens drawn from sources such as Vicipaedia, OSCAR, the Latin Library, Latin Universal Dependency treebanks, and cc100-latin, among others.[8]

Analogy Resolution Dataset Development[9]

For the first version of this evaluation dataset, twelve analogy categories were proposed to cover some common syntactic and semantic relationships. In this respect, the categorization follows organization of Mikolov et al. 2013 (cf. Table 1): we try first to repurpose syntactic-relationship analogies, e.g. "superlative" and "plural verbs" as well as semantic-relationship analogies, e.g. "man-woman." Not all categories translate well to the language and domain; for example, "common capital city," despite often being the go-to example for demonstrating the efficacy of vector models, cannot be reasonably applied to Latin historical text data. Other categories, e.g. "plural verbs" are underdescribed in the original paper and are adapted here specifically as singular-plural pairs of 3rd person present indicative active verbs. There are undoubtedly more

[7] Some relevant literature on the question of word embeddings evaluation, including work with attention to low-resource and ancient languages, can be found at Schnabel et al. 2015; Stringham and Izbicki 2020; Coto-Solano 2022; Dereza et al. 2024.

[8] Vicipaedia: https://la.wikipedia.org/wiki/Vicipaedia; OSCAR: https://oscar-project.org/; Latin Library: https://github.com/cltk/lat_text_latin_library; Universal Dependencies: https://github.com/UniversalDependencies; cc100-latin: https://huggingface.co/datasets/pstroe/cc100-latin.

[9] The basic work of identifying categories useful for evaluation as well as bootstrapping the word lists per category was completed done by William Bramwell in summer 2024 as part of an internship in the Institute for the Study of the Ancient World Library under my supervision. These categorical word lists are the basis of both the analogy and odd-one-out datasets described in the report.

categories to which a Latin-specific evaluation dataset could be expanded to; it is recommended that this be the subject of future work.

The syntactic categories are:

- Verbs: singular to plural, e.g. *dat* → *dant*
- Verbs: 3rd to 1st person, e.g. *dat* → *do*
- Verbs: present to imperfect, e.g. *dat* → *dabat*
- Verbs: present to perfect, e.g. *dat* → *dedit*
- Nouns: singular to plural, e.g. *homo* → *homines*
- Adjectives: positive to comparative, e.g. *bonus* → *melior*
- Adjectives: positive to superlative, e.g. *bonus* → *optimus*
- Adjectives to adverbs, e.g. *bonus* → *bene*

The semantic categories are:

- Adjectives: opposites, e.g. *bonus* → *malus*
- Nouns: masculine to feminine, e.g. *rex* → *regina*
- Nouns: familial relationships, e.g. *pater* → *mater*

Each category list begins with attention to the most frequently occurring words for each part-of-speech tag as given in the Dickinson Core Latin Vocabulary (Francese 2021). The lists are then curated by hand to remove a variety of anomalies. To give just a few examples:

1. *sequor* is removed from "Verbs: present to perfect" since the perfect form of the verb consists of two words (*secutus est*) and the analogy evaluation task assumes single word resolutions;
2. *manus* is removed from "Nouns: singular to plural" since the plural form (that is, without macrons or some other indicator of vowel length) is a homograph of the singular form and the analogy evaluation task assumes that the correct resolution cannot be the same as its analogical pairing.

Some lists like the one for "Nouns: Masculine to feminine" have been supplemented by the authors to ensure that there are enough examples even from words that are not in the Core vocabulary (e.g. *magister* and *magistra*, the former in the Core vocabulary, the latter not). There is one further check on the word pairs themselves; we check that both words in every pair are included in the top 25,000 Latin word forms as drawn from LatinCy Assets with the goal of not evaluating any analogies that would consist of particularly infrequent words. The contents of all lists are available in the evaluation dataset repository.[10]

The analogies themselves are then built category-wise from combinations of these list items. That is, for example, with the first three word pairs from the "Verbs: singular to plural" category being *est*:*sunt*, *potest*:*possunt*, *dat*:*dant*, analogies such as *est*:*sunt*::*potest*:*possunt*, *est*:*sunt*::*dat*:*dant*, *potest*:*possunt*::*dat*:*dant* are created. The resulting base analogies evaluation dataset consists of 1,383 analogies.[11]

Odd-one-out Selection Dataset Development

A second evaluation dataset has been developed for the LatinCy w2v models, further making use of the categories described in the previous section. For this dataset, four words are chosen from one half of word pairs and a random fifth word is chosen from the second half of the word pairs. For example, using the category "Verbs: singular to plural," the selection process may first take the four words *est*, *potest*, *dat*, and *videt* from the first halves of the categorical word pairs and then a fifth word *faciunt* at random from second halves. The evaluation logic is that the first four words should retain within the LatinCy vectors a strong enough "singular" signal than that of the plural verb *faciunt* should be returned as the odd-one-out using the Gensim `doesnt_match` method (details below). The combinatorial possibilities for this dataset expansion are quite large, so the number of generated one-one-out instances has been artificially constrained: again, word pairs are only drawn from high frequency vocabulary (here, the top 5,000 words), and furthermore, categories were limited to a maximum of 500 combinations, while categories that

[10] https://github.com/diyclassics/latincy-w2v-eval/tree/main/data/analogy-types.
[11] https://github.com/diyclassics/latincy-w2v-eval/blob/main/data/evaluation-datasets/latin_analogies_generated.txt.

failed to yield at least 5 combinations were excluded. The resulting base odd-one-out evaluation dataset consists of 2,729 items.[12]

Evaluation Methodology

Both the analogies and odd-one-out tasks are evaluated using Gensim methods.

Analogy evaluation is computed with the Gensim `most_similar` method with the positive and negative terms set up as follows, assuming an analogy of the form w1:w2::w3:w4: `model.most_similar(positive=[w2, w3], negative=[w1])`. Moreover, the top 10 "most similar" vector results are returned. We then see at which rank, if any, w4 matches the output of the `most_similar` call. We then compute accuracy as number of correctly solved analogies at each rank over the total number of analogies. One constraint on this accuracy computation is that we discard any analogy for which no available model can return a correctly solved analogy. We take this measure so as not to penalize the models for any analogies generated by the combinatorial expansion that are either malformed or unduly challenging or even just too difficult for the quality of the current models. As the models improve, it is expected that fewer items will need to be removed in this manner.

Odd-one-out evaluation is computed with the Gensim `doesnt_match` method with the parameters given as follows, assuming a data point of the form w1, w2, w3, w4, w5 where w5 is the confounder: `model.doesnt_match([w1, w2, w3, w4, w5])`. We then see which for which items does the function correctly return the fifth term, computing accuracy as number of correctly solved odd-one-out calls over the total number of analogies. The same constraint on solvability noted above for analogies is used as well for the odd-one-out task.

Results / Findings

Table 1 shows the results of the analogies task.

[12] https://github.com/diyclassics/latincy-w2v-eval/blob/main/data/evaluation-datasets/latin_oddoneout_generated.tsv.

Model	Rank 1	Rank 2	Rank 3	Rank 5	Rank 10
latincy_w2v-CBOW-50-10-0_0_2	0.542	0.675	0.749	0.830	0.953
latincy_w2v-CBOW-50-5-0_0_2	0.545	0.690	0.771	0.864	0.979
latincy_w2v-SG-50-5-0_0_2	0.525	0.664	0.757	0.831	0.964
latincy_w2v-SG-50-10-0_0_2	0.506	0.667	0.764	0.839	0.946
latincy_w2v-CBOW-100-10-0_0_2	0.716	0.855	0.949	**1.000**	**1.000**
latincy_w2v-CBOW-100-5-0_0_2	0.705	0.853	0.940	**1.000**	**1.000**
latincy_w2v-SG-100-5-0_0_2	0.701	0.865	0.955	**1.000**	**1.000**
latincy_w2v-SG-100-10-0_0_2	0.675	0.819	0.884	0.978	**1.000**
latincy_w2v-CBOW-300-10-0_0_2	**0.766**	**0.938**	**1.000**	**1.000**	**1.000**
latincy_w2v-CBOW-300-5-0_0_2	0.752	0.920	0.999	**1.000**	**1.000**
latincy_w2v-SG-300-10-0_0_2	0.584	0.754	0.854	0.988	**1.000**
latincy_w2v-SG-300-5-0_0_2	0.565	0.775	0.881	**1.000**	**1.000**

Table 2: Accuracy of solving analogies for the LatinCy word2vec models at various ranks; note that of the original 1,383 analogies included in the evaluation dataset, 800 are able to be solved by at least one of the models shown here. So, for example, looking at *latincy_w2v-CBOW-300-10*, this model correctly solves 613 of the 800 solvable analogies, i.e. 76.6%

The best performing model on the analogies task is *latincy_w2v-CBOW-300-10* which is able to correctly solve 76.6% of solvable analogies at rank 1, 93.8% at rank 2, and 100% starting with rank 3. It is worth noting that another 300-length vector set, i.e. *latincy_w2v-CBOW-300-5*, demonstrates only slightly worse performance. Note too that both the 100- and 300-length vectors are just about all correctly solving 100% of the solvable analogies at rank 5.

Table 2 shows the results of the odd-one-out task.

Model	Accuracy
latincy_w2v-CBOW-100-10-0_0_2	0.626
latincy_w2v-CBOW-100-5-0_0_2	0.642
latincy_w2v-CBOW-300-10-0_0_2	0.666
latincy_w2v-CBOW-300-5-0_0_2	**0.667**
latincy_w2v-CBOW-50-10-0_0_2	0.620
latincy_w2v-CBOW-50-5-0_0_2	0.613
latincy_w2v-SG-100-10-0_0_2	0.543
latincy_w2v-SG-100-5-0_0_2	0.556
latincy_w2v-SG-300-10-0_0_2	0.562
latincy_w2v-SG-300-5-0_0_2	0.577
latincy_w2v-SG-50-10-0_0_2	0.549
latincy_w2v-SG-50-5-0_0_2	0.560

Table 2: Accuracy of the odd-one-out task for the LatinCy word2vec models; note that of the original 2,728 items included in the evaluation dataset, 2,179 are able to be solved by at least one of the models shown here. So, for example, looking at *latincy_w2v-CBOW-300-5*, this model correctly solves 1,820 of the 2,179 solvable analogies, i.e. 66.7%.

The best performing model on the odd-one-out task is *latincy_w2v-CBOW-300-5* which is able to correctly solve 66.7% of solvable items. As with the analogies task, the other 300-length vector set, i.e. *latincy_w2v-CBOW-300-10*, demonstrates almost identical performance. Another thing worth noting here is that the difference between skipgram and the CBOW models is more apparent with this task than with the analogies task; the worst performing CBOW model (61.3%) still shows a higher accuracy than the best performing skipgram model (57.7%).

Conclusion

In this report, we present a starting point for renewed interest in Latin static word embeddings evaluation and one specifically that can be used as a benchmark and a evaluative framework within the LatinCy project. The available next steps are numerous. As noted above, iterative development of the models along with iterative development, expansions, and refinement of the

evaluation datasets. More categories, more words, more overall evaluation items in total will all serve the community well in working towards increasingly powerful and generalizable vector-space models for Latin.

In this initial foray into evaluation dataset development, the compilation of syntactic categories (e.g. "Verbs: singular to plural" or "Verbs: 3rd to 1st person") was more straightforward than that of semantic categories (e.g. "Nouns: familial relationships"). Additional attention should be paid to semantic coverage when expanding the evaluation dataset.

More attention in future work should also be paid to relative success and weakness within different categories. For example, looking at the best performing model on analogies (*latincy_w2v-CBOW-300-10*), accuracy on the category "Verbs: singular to plural" is 91.9% where as "adjective to adverb" is only 9.7%! Accordingly, it is critical for Latin computational philologists to drill down on subsets of embeddings performance to better understand how, when, or even if they should be used for a specific downstream task.

Another area for future work should be expanding the number and kinds of evaluation tasks. Evaluating the models through performance on downstream tasks such as lemmatization and part-of-speech tagging is one direction. Another is to follow strategies used successfully for other languages, such as those, for example, summarized and discussed in Schnabel et al. 2015.

Lastly, we have confined ourselves in this report to evaluation of word2vec models. But as part of the LatinCy project in general, we have also developed other static models, like GLoVe vectors, and subword models, like fasttext and floret vectors. These will be added to models comparisons in future work.

Works Cited

Bjerva, Johannes, and Praet, Raf. 2015. "Word Embeddings Pointing the Way for Late Antiquity." In *Proceedings of the 9th SIGHUM Workshop on Language Technology for Cultural Heritage, Social Sciences, and Humanities*: 53–57.

Burns, Patrick J. 2023. "LatinCy: Synthetic Trained Pipelines for Latin NLP." https://arxiv.org/abs/2305.04365v1.

———. forthcoming. "Pretrained Word Vectors for Latin Philology."

Burns, Patrick J., Brofos, James A., Li, Kyle, Chaudhuri, Pramit, and Dexter, Joseph P. 2021. "Profiling of Intertextuality in Latin Literature Using Word Embeddings." In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* 4900–4907. Online: Association for Computational Linguistics. https://www.aclweb.org/anthology/2021.naacl-main.389.

Coto-Solano, Rolando. 2022. "Evaluating Word Embeddings in Extremely Under-Resourced Languages: A Case Study in Bribri." In edited by Nicoletta Calzolari, Chu-Ren Huang, Hansaem Kim, James Pustejovsky, Leo Wanner, Key-Sun Choi, Pum-Mo Ryu, et al.4455–67. Gyeongju, Republic of Korea: International Committee on Computational Linguistics. https://aclanthology.org/2022.coling-1.393.

Dereza, Oksana, Doyle, Adrian, Rani, Priya, Ojha, Atul Kr, Moran, Pádraic, and McCrae, John P. 2024. "ACHILLES: Ancient and Historical Language Evaluation Set: The Dataset for SIGTYP 2024 Shared Task on Word Embedding Evaluation for Ancient and Historical Languages." https://zenodo.org/doi/10.5281/zenodo.10655061.

Francese, Christopher. 2021. "Latin Core Vocabulary." *Dickinson College Commentaries* . https://dcc.dickinson.edu/vocab/core-vocabulary.

Honnibal, Matthew, and Montani, Ines. 2023. "spaCy: Industrial-Strength Natural Language Processing in Python." https://spacy.io/.

Manjavacas, Enrique, Long, Brian, and Kestemont, Mike. 2019. "On the Feasibility of Automated Detection of Allusive Text Reuse." *arXiv Preprint arXiv:1905.02973* .

Mikolov, Tomas, Chen, Kai, Corrado, Greg, and Dean, Jeffrey. 2013. "Efficient Estimation of Word Representations in Vector Space." arXiv:1301.3781v3 [cs.CL]: arXiv. https://arxiv.org/abs/1301.3781.

Schnabel, T., Labutov, I., Mimno, D., and Joachims, T. 2015. "Evaluation Methods for Unsupervised Word Embeddings." In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing* 298–307. https://aclanthology.org/D15-1036.pdf.

Sprugnoli, Rachele, Moretti, Giovanni, and Passarotti, Marco. 2020. "Building and Comparing Lemma Embeddings for Latin: Classical Latin versus Thomas Aquinas." *Italian Journal of Computational Linguistics* 6.1: 29–45.

Sprugnoli, Rachele, Passarotti, Marco, and Moretti, Giovanni. 2019. "*Vir* Is to *Moderatus* as *Mulier* Is to *Intemperans* Lemma Embeddings for Latin." In *Proceedings of the Sixth Italian Conference on Computational Linguistics* 1–7. Bari, Italy. http://ceur-ws.org/Vol-2481/paper69.pdf.

Stringham, Nathan, and Izbicki, Mike. 2020. "Evaluating Word Embeddings on Low-Resource Languages." In *Proceedings of the First Workshop on Evaluation and Comparison of NLP Systems* 176–86. Online: Association for Computational Linguistics. https://aclanthology.org/2020.eval4nlp-1.17.